

Information Retrieval

Weike Pan

The slides are **adapted from those provided by Prof. Hinrich Schütze** at University of Munich (<http://www.cis.lmu.de/~hs/teach/14s/ir/>).

Chapter 5 Index compression

- 5.1 Statistical properties of terms in information retrieval
- 5.2 Dictionary compression
- 5.3 Postings file compression
- 5.4 References and further reading

Outline

- 5.1 Statistical properties of terms in information retrieval
- 5.2 Dictionary compression
- 5.3 Postings file compression
- 5.4 References and further reading

5.1 Statistical properties of terms in information retrieval

- Reuters RCV1 statistics
 - documents: $N=800,000$
 - tokens per document: $L=200$
 - terms: $M=400,000$
 - bytes per token (including spaces/punctuations): 6
 - bytes per token (without spaces/ punctuations): 4.5
 - bytes per term: 7.5
 - non-positional postings: $T=100,000,000$

5.1 Statistical properties of terms in information retrieval

Heaps' law

- How many distinct words are there? Can we assume there is an upper bound? The vocabulary will keep growing with the collection size.
- **Heaps' law:** $M = kT^b$
 - M is the vocabulary size, T is the number of tokens in the collection.
 - Typical values for the parameters k and b are: $30 \leq k \leq 100$ and $b \approx 0.5$.
 - It is the simplest possible relationship between the collection size T and the vocabulary size M in the log-log space. It is an empirical law.
- **Question:** Can you verify the Heaps' law using the RCV1 data? (Plot the statistics in a $\log_{10}M$ - $\log_{10}T$ space)

5.1 Statistical properties of terms in information retrieval

Zipf's law

- How many frequent vs. infrequent terms we should **expect** in a collection?
- **Zipf's law:** cf_i is the collection frequency of **the i -th most frequent term** t_i , i.e., the number of occurrences of the term t_i in the collection.
 - cf_i **is proportional to** $\frac{1}{i}$
- Question: Can you verify Zipf's law using the RCV1 data?

Outline

- 5.1 Statistical properties of terms in information retrieval
- 5.2 Dictionary compression
- 5.3 Postings file compression
- 5.4 References and further reading

5.2 Dictionary compression

- The dictionary is small compared with the postings file (i.e., postings lists).
- But we want to **keep it in memory**.
- Also: competition with other applications, cell phones, onboard computers, fast startup time. Hence, **compressing the dictionary is important**.

5.2 Dictionary compression

Dictionary as an array of fixed-width entries (1/2)

term	document frequency	pointer to postings list
a	656,265	→
aachen	65	→
...	...	...
zulu	221	→

space needed: 20 bytes 4 bytes 4 bytes

- Space for RCV1 data: $(20+4+4)*400,000 = 11.2 \text{ MB}$

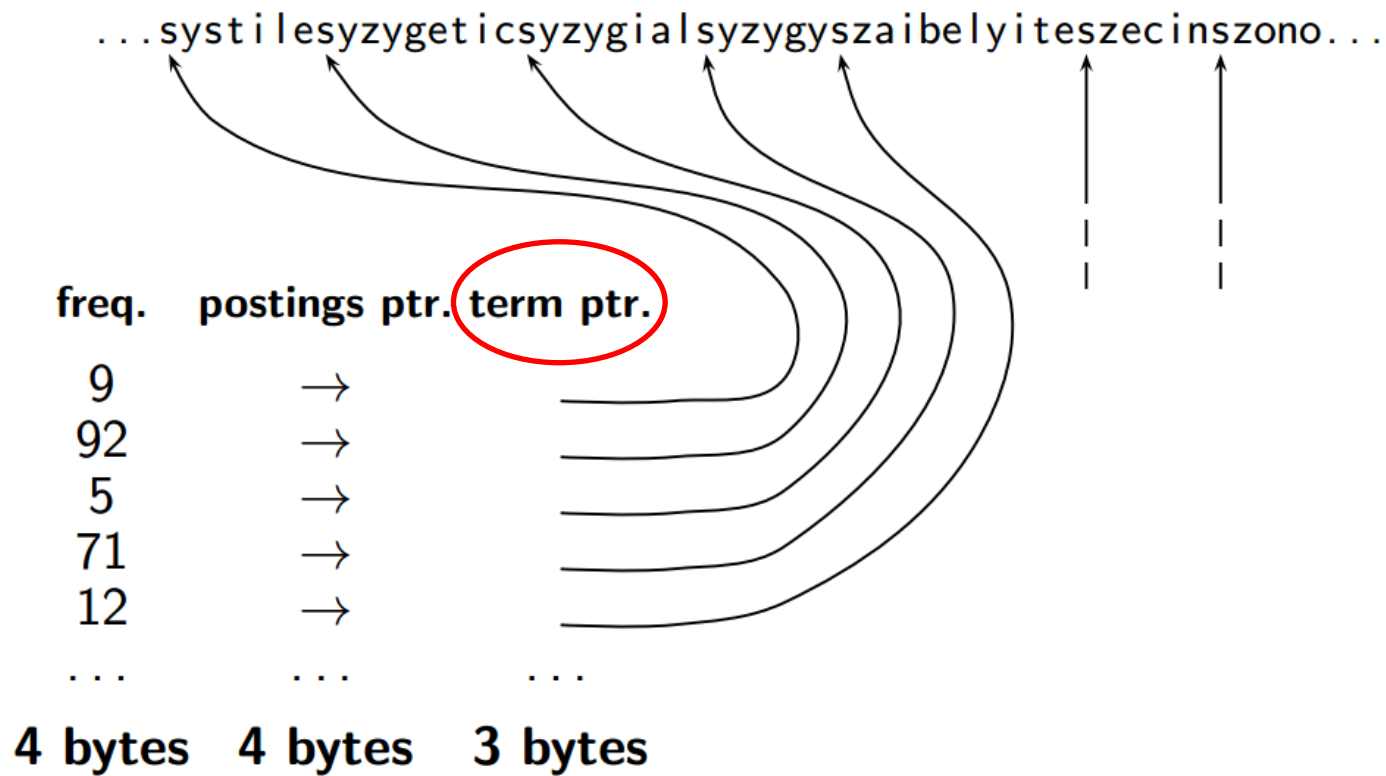
5.2 Dictionary compression

Dictionary as an array of fixed-width entries (2/2)

- Most of the bytes in the term column are **wasted**. We allot (分配) 20 bytes for terms of length 1.
- But, we **still can not handle** hydrochlorofluorocarbons and supercalifragilisticexpialidocious.
- Average length of a term in English: 8 characters (or a little bit less)
- How can we use "on average 8 characters per term"?

5.2 Dictionary compression

Dictionary as a **string** (1/2)



5.2 Dictionary compression

Dictionary as a **string** (2/2)

- 4 bytes per term for **document frequency**
- 4 bytes per term for **pointer to postings list**
- 8 bytes (on average) for **term** in string
- **3 bytes** per pointer into string (needs $\log_2(8 * 400000) = 21.6 < 24$ **bits** to resolve $8 * 400000$ positions)
- Space: $400000 * (4 + 4 + 8 + 3) = 7.6\text{MB}$ (compared to 11.2 MB for fixed-width array)

5.2 Dictionary compression

Dictionary as a string with **blocking** (1/2)

...7systile9syzygetic8syzygial6syzygy11szaibelyite6szecin...

freq.	postings ptr.	term ptr.
-------	---------------	-----------

9

→

92

→

5

→

71

→

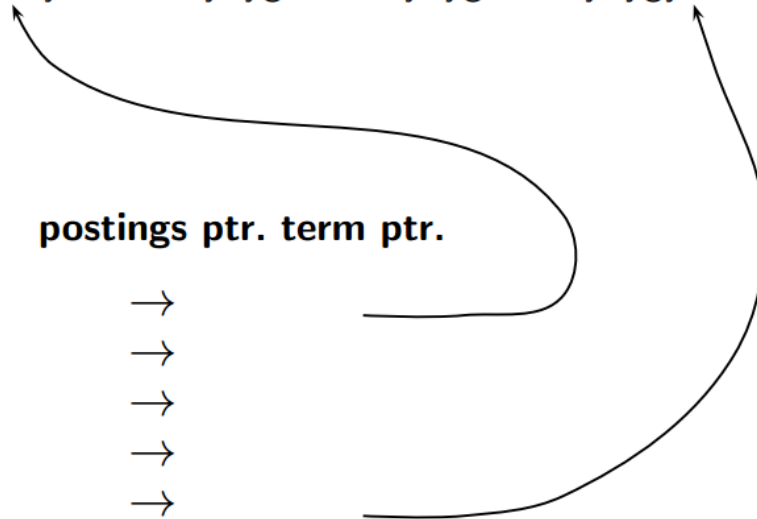
12

→

...

...

...



5.2 Dictionary compression

Dictionary as a string with **blocking** (2/2)

- Example: block size $k = 4$
 - we use 4×3 bytes for term pointers without blocking ...
 - we now use **3 bytes** for one pointer plus **4 bytes** for indicating the length of each term.
 - We **save $12 - (3 + 4) = 5$ bytes per block.**
 - Total savings: $400000/4 * 5 =$ **0.5 MB**
- This reduces the size of the dictionary from 7.6 MB to **7.1 MB**.

5.2 Dictionary compression

Front coding

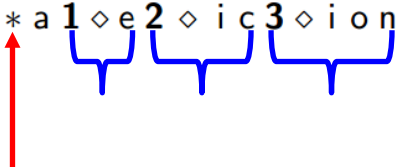
One block in blocked compression ($k = 4$) ...

8 a u t o m a t a **8** a u t o m a t e **9** a u t o m a t i c **10** a u t o m a t i o n



... further compressed with front coding.

8 a u t o m a t * a **1** ◇ e **2** ◇ i c **3** ◇ i o n

A diagram illustrating front coding. A red arrow points to the asterisk between 't' and 'a'. Blue brackets are placed under the words 'e', 'ic', and 'ion' in the subsequent words, indicating that these words are being referenced or compressed.

5.2 Dictionary compression

Dictionary compression for the RCV1 data: Summary

data structure	size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
~, with blocking, $k = 4$	7.1
~, with blocking & front coding	5.9

Outline

- 5.1 Statistical properties of terms in information retrieval
- 5.2 Dictionary compression
- 5.3 Postings file compression
- 5.4 References and further reading

5.3 Postings file compression

- The postings file (i.e., postings list) is much larger than the dictionary: a factor of at least 10.
- A posting for our purpose is a **docID**.
- For the RCV1 data (800000 documents), we would use **32 bits** per docID when using 4-byte integers.
- Alternatively, we can use $\log_2 \underline{800000} \approx 19.6 < \mathbf{20 \text{ bits}}$ per docID.
- Our goal: use **a lot less than 20 bits** per docID.

5.3 Postings file compression

Key idea: Store **gaps instead of docIDs**

- Each postings list is ordered **in increasing order** of docID.
 - Example postings list: computer: 283154, 283159, 283202, ...
- It suffices to store **gaps**: $283159 - 283154 = 5$, $283202 - 283159 = 43$
- Example postings list using gaps: computer: 283154, **5**, **43**, ...
- Gaps for frequent terms are **small**.
- Thus: We can **encode small gaps** with **fewer than 20 bits**.

5.3 Postings file compression

Gap encoding

	encoding	postings list					
THE	docIDs	...	283042	283043	283044	283045	...
	gaps		1	1	1		...
COMPUTER	docIDs	...	283047	283154	283159	283202	...
	gaps		107	5	43		...
ARACHNOCENTRIC	docIDs	252000	500100				
	gaps	252000	248100				

5.3 Postings file compression

Variable length encoding

- Aim:
 - For ARACHNOCENTRIC and other **rare** terms, we will use **about 20 bits per gap**.
 - For THE and other very **frequent** terms, we will use **only a few bits per gap**.
- In order to implement this, we need to devise some form of variable length encoding
 - **Variable length encoding** uses few bits for small gaps and many bits for large gaps.

5.3 Postings file compression

Variable byte (VB) code

- Used by many commercial/research systems
- Dedicate one bit (high bit) to be **a continuation bit** c .
 - **If** the gap G fits within 7 bits, binary-encode it in the 7 available bits and set **$c = 1$** .
 - **Else**: encode the lower-order 7 bits and then use one or more additional bytes to encode the higher order bits using the same algorithm.
- At the end, set the **continuation bit** of **the last byte to 1 ($c = 1$)** and of **the other bytes to 0 ($c = 0$)**.

5.3 Postings file compression

VB code encoding algorithm

<u>VBENCODENUMBER(<i>n</i>)</u>	<u>VBENCODE(<i>numbers</i>)</u>
1 <i>bytes</i> $\leftarrow \langle \rangle$	1 <i>bytestream</i> $\leftarrow \langle \rangle$
2 while <i>true</i>	2 for each <i>n</i> $\in$ <i>numbers</i>
3 do <u>PREPEND(<i>bytes</i>, <i>n</i> mod 128)</u>	3 do <i>bytes</i> $\leftarrow$ <u>VBENCODENUMBER(<i>n</i>)</u>
4 if <i>n</i> < 128	4 <i>bytestream</i> $\leftarrow$ EXTEND(<i>bytestream</i> , <i>bytes</i>)
5 then BREAK	5 return <i>bytestream</i>
6 <i>n</i> $\leftarrow$ <i>n</i> div 128	
7 <u><i>bytes</i>[LENGTH(<i>bytes</i>)] += 128</u>	
8 <u>return <i>bytes</i></u>	

 continuation bit (c=1)

- PREPEND: adds an element to the beginning of a list,
e.g., PREPEND(< 2 >, 1) = < 1, 2 >.
- If *n*=130, we have 00000001 10000010.
- EXTEND: extends a list, e.g., EXTEND(<1,2>, <3,4>) = <1,2,3,4>.

5.3 Postings file compression

VB code decoding algorithm

```
VBDECODE(bytestream)
1  numbers  $\leftarrow \langle \rangle$ 
2  n  $\leftarrow 0$ 
3  for i  $\leftarrow 1$  to LENGTH(bytestream)
4  do if bytestream[i] < 128
5      then n  $\leftarrow 128 \times n + \textit{bytestream}[i]$ 
6      else n  $\leftarrow 128 \times n + (\textit{bytestream}[i] - 128)$ 
7          APPEND(numbers, n)
8          n  $\leftarrow 0$ 
9  return numbers
```

Diagram illustrating the VB code decoding algorithm:

- Line 4: *bytestream*[*i*] < 128 is checked. A purple arrow points from this condition to a green box labeled "continuation bit (c=0)".
- Line 6: (*bytestream*[*i*] - 128) is calculated. A red arrow points from this expression to a green box labeled "continuation bit (c=1)".

5.3 Postings file compression

Example

docIDs	824	829	215406
gaps		5	214577
VB code	<u>00000110 10111000</u>	<u>10000101</u>	<u>00001101 00001100 10110001</u>



continuation bit (c=1)

5.3 Postings file compression

Other variable codes

- Instead of bytes, we can also use a different “unit of alignment”: 32 bits (words), 16 bits, 4 bits (nibbles), etc
- Variable byte alignment wastes space if you have many small gaps - nibbles do better on those.

5.3 Postings file compression

Gamma codes for gap encoding

- [illegible]

5.3 Postings file compression

Gamma code

- Represent a gap G as a pair of **length** and **offset**.
 - Offset is the gap in binary, **with the leading bit chopped off**, e.g., 13→1101→**101** = offset.
 - Length is the length of offset, e.g., for 13 (the offset is 101), this is 3. Encode length in **unary** code: **1110**.
 - Gamma code of 13 is the concatenation of length and offset: **1110101**.

5.3 Postings file compression

length is the length of offset



Gamma code examples

number	unary code	length	offset	γ code
0	0			
1	10	0		0
2	110	10	0	10,0
3	1110	10	1	10,1
4	11110	110	00	110,00
9	111111110	1110	001	1110,001
13		1110	101	1110,101
24		11110	1000	11110,1000
511		111111110	11111111	111111110,11111111
1025		1111111110	0000000001	1111111110,0000000001

5.3 Postings file compression

Length of gamma code

- The length of **offset** is $\lceil \log_2 G \rceil$ bits.
- The length of **length** is $\lceil \log_2 G \rceil + 1$ bits,
- So the length of the entire code is $2 * \lceil \log_2 G \rceil + 1$ bits.
- Gamma codes are always of odd length (奇数长度).
- Gamma codes are within a factor of 2 of the optimal encoding length $\log_2 G$.

5.3 Postings file compression

Gamma code: Properties

- Gamma code (like variable byte code) is **prefix-free**: a valid code word is not a prefix of any other valid code.
- Gamma code is **parameter-free**.
- More theoretical analysis on the compression rate can be found in the textbook.

5.3 Postings file compression

Gamma codes: Alignment

- Machines have word boundaries, e.g., 8, 16, 32 bits.
- Compressing and manipulating at granularity of bits can be slow.
- Variable byte encoding is aligned and thus potentially more efficient.
- Regardless of efficiency, variable byte is conceptually simpler at little additional space cost.

5.3 Postings file compression

Compression of the RCV1 data

data structure	size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
~, with blocking, $k = 4$	7.1
~, with blocking & front coding	5.9
collection (text, xml markup etc)	3600.0
collection (text)	960.0
T/D incidence matrix	40,000.0
postings, uncompressed (32-bit words)	400.0
postings, uncompressed (20 bits)	250.0
postings, variable byte encoded	116.0
postings, γ encoded	101.0

5.3 Postings file compression

Summary

- We can now create an index for highly efficient Boolean retrieval that is **very space efficient**.
- Only 10-15% of the total size of the text in the collection.
- However, we've ignored **positional** and **frequency** information.
- For this reason, space savings are less in reality.

Summary

- 5.1 Statistical properties of terms in information retrieval
- 5.2 Dictionary compression
- 5.3 Postings file compression
- 5.4 References and further reading